

ПРИМЕРИ С ЦИКЛИ

Всичко, което може да се напише с цикъл `for()`, може да се напише и с `while()`.

```
Console.WriteLine("=== FOR ЦИКЪЛ ===");
for (int i = 0; i < 5; i++)
{
    Console.WriteLine($"i = {i}");
}

Console.WriteLine("\n=== WHILE ЦИКЪЛ ===");
int j = 0;
while (j < 5)
{
    Console.WriteLine($"j = {j}");
    j++;
}
```

Използвайте FOR когато:

- Знаете точния брой итерации предварително
- Работите с колекции по индекс
- Нуждаете се от брояч

```
int[] array = new int[5];
for (int i = 0; i < array.Length; i++)
{
    Console.WriteLine(array[i]);
}
```

Използвайте WHILE когато:

- Не знаете колко итерации ще са нужни
- Условието зависи от външни фактори
- Четете от поток/файл до край

```
// X ОПАСНО - може да стане безкраен цикъл
int x = 0;
while (x < 10)
{
    Console.WriteLine(x);
    // Забравихме x++ - БЕЗКРАЕН ЦИКЪЛ!
}
```

Условието винаги е вярно и отпечатва безброй 0-и. Какво ще стане, ако условието е грешно? Не влиза изобщо в цикъла и го прескача.

```
int x = 0;

while (x > 1) не е вярно,
{           не влиза в цикъла
    Console.WriteLine(x);
    //x++;
}
Console.WriteLine("Прескача");
```

Критерий	FOR	WHILE
Предварителен брой итерации	✓ Идеален	✗ Неефикасен
Неизвестен брой итерации	✗ Неудобен	✓ Идеален
Риск от безкраен цикъл	Нисък	Висок
Четимост	Отлично за индекси	Добро за условия

Между скобите на Parse() може да се вкара променлива, която е от текстов тип string, а тогава Parse(input) ще бъде преобразувана в число тип int в нашия пример. Четем текст string, ако той не е „stop” го считаме за число и го парсваме до int. Има и друг метод TryParse, който обаче връща булева стойност True/False дали стрингът може да се конвертира в число. Може да използваме и друга променлива за средна стойност тип double, която наричаме average, но напомним, че понеже имаме целочислено деление на 2 променливи тип int / int, дробната част се отрязва при

това положение, въпреки че резултатът ще бъде присвоен на тип double, затова, непосредствено преди делението, правим явно (експлицитно) конвертиране (double).

```
string input = Console.ReadLine().ToLower(); | чете от конзолата
int sum = 0, count = 0; | прави всичко в малки букви
double average = 0; | заделяме памет

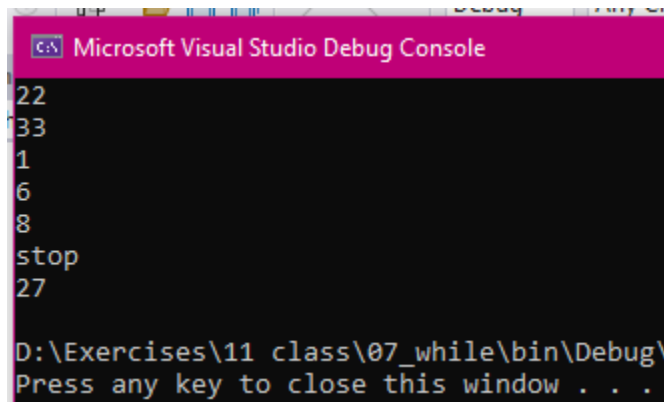
1) условие за край
while(input != "stop") | с малки букви избягваме || "STOP" || "Stop" проверки
{
    int n = int.Parse(input); | щом не е текст, считаме го за число

    3) среда
    if( n % 11 == 0) | някакво условие, което няма значение в
    { | случая и е според задачата.
        sum += n; | гледаме винаги сумата да е преди брояча!
        count++;
    }

    2) стъпка за продължаване преди }
    input = Console.ReadLine().ToLower(); | счита ги за малки букви
    | int / int
    average = (double) sum / count;
    Console.WriteLine(average); | 4) отпечатване
}
```

Демонстрация без (double):

```
average = sum / count;
Console.WriteLine(average);
```



```
Microsoft Visual Studio Debug Console
22
33
1
6
8
stop
27

D:\Exercises\11 class\07_while\bin\Debug\
Press any key to close this window . . .
```

$22 + 33 = 55 / 2 = 27.5$, но дробната част се отрязва. Затова винаги трябва да се застраховаме и слагаме (double) при средна стойност и като цяло внимаваме при деление. Останалите числа се пренебрегват, понеже не са кратни на 11. Нека сега добавим (double)

```
average = (double) sum / count;
Console.WriteLine(average);
```

```
Microsoft Visual Studio Debug Console
22
33
4
stop
27.5
D:\Exercises\11 class\07_while\bin\Debug\n
Press any key to close this window . . .
```

- Използвайте **FOR** когато знаете броя итерации
- Използвайте **WHILE** когато условието е динамично
- Винаги тествайте условията на прекратяване
- Ограничавайте максималния брой итерации

Задача за упражнение и оценка. Напишете програма, която чете въведеното от конзолата, докато не й се подаде команда „END”, а ако срещне числа, кратни на 7, пресмята квадрата им. Примерно решение:

```
string input = Console.ReadLine().ToUpper();
// в главни букви

while (input != "END")
{
    int n = int.Parse(input);

    if (n % 7 == 0)
    {
        Console.WriteLine($"{n}^2 = {n*n}");
    }

    input = Console.ReadLine().ToUpper();
}
```

```
Microsoft Visual Studio Debug Console
4
7
7^2 = 49
12
14
14^2 = 196
end

D:\Exercises\11 class\07_while\bin\Debug\n
Press any key to close this window . . .
```

Има още една основна концепция в програмирането, и тя е, че много пъти, представянето е в обратен ред, спрямо това, с което сме свикнали в математиката. Например, за вариациите на топки 6 от 49 -> за първата топка имаме 49 възможни изтегляния, за втората -> 48, трета -> 47 и т.н. за последната 6-та топка остават 44 начина. Това се реализира алгоритмично така:

```
✓// Всички вариации без повторения на 6 от 49 – подредбата има значение.
|// При комбинациите редът няма значение и се брои по веднъж.
```

```
✓for (int i1 = 1; i1 <= 44; i1++)
|{
|  ✓for (int i2 = i1 + 1; i2 <= 45; i2++)
|  |{
|  |  ✓for (int i3 = i2 + 1; i3 <= 46; i3++)
|  |  |{
|  |  |  ✓for (int i4 = i3 + 1; i4 <= 47; i4++)
|  |  |  |{
|  |  |  |  ✓for (int i5 = i4 + 1; i5 <= 48; i5++)
|  |  |  |  |{
|  |  |  |  |  ✓for (int i6 = i5 + 1; i6 <= 49; i6++)
|  |  |  |  |  |{
|  |  |  |  |  |  Console.WriteLine(i1 + " " + i2 + " " +
|  |  |  |  |  |  |  i3 + " " + i4 + " " + i5 + " " + i6);
|  |  |  |  |  |  }
|  |  |  |  |  }
|  |  |  |  }
|  |  |  }
|  |  }
|  }
|}
|}
```

Ако в математиката сме свикнали да решаваме дадена задача в определен ред, същата задача се имплементира (реализира) на програмен език, обикновено в обратен ред. Това трябва да го имаме като идея. Пример е изреждането на тези топки за тото 6 от 49: за първа топка имаме 49 възможности, а за шестата – 45, но както виждаме, понеже първо се върти

цикълът на най-вътрешния for(), а последен – най-външния, влагането им е наобратно при самото писане и четене.

Може би ще стане по-ясно, когато се даде друг нагледен пример за изчисляване на факториел. От математиката сме свикнали $n! = 1.2.3.4... (n-2).(n-1).n$, но от гледна точка на програмирането, реализацията често е в обратна посока: $n.(n-1).(n-2)...3.2.1$

```
// n! = n *(n-1)*(n-2)*(n-3)*...*2*1
int n = int.Parse(Console.ReadLine());

decimal factorial = 1; // произведение да не започва с 0

// Perform an "infinite loop"
while (n>1) // иначе става 0 *...
{
    factorial *= n;
    n--;
}
Console.WriteLine("n! = " + factorial);
```

Ако не сме сигурни как се променят стойностите, пускаме дебъгера и ги проследяваме с F11:

The screenshot shows a Visual Studio IDE with a C# program for calculating factorial. A red dot indicates a breakpoint is set at line 45, which is `factorial *= n;`. The code is as follows:

```
37 // n! = n *(n-1)*(n-2)*(n-3)*...*2*1
38 int n = int.Parse(Console.ReadLine());
39
40 decimal factorial = 1;
41
42 // Perform an "infinite loop"
43 while (n>1)
44 {
45     factorial *= n;
46     n--;
47 }
48 Console.WriteLine("n! = " + factorial);
49 Console.WriteLine($"В момента е {DateTime.Now}");
```

The debugger is active, and the Autos window at the bottom shows the current state of variables:

Name	Value	Type
factorial	5	decimal
n	4	int

The Call Stack window on the right shows the current method call: `07_while.dll!Program.Main`.

Втори начин – с do-while си застраховаме поне едно изпълнение. Използваме тип BigInteger, за да проверим за много големи числа като $n = 100$:

```
using System.Numerics;

Console.Write("n = "); за n = 100
int n = int.Parse(Console.ReadLine());
BigInteger factorial = 1;
do
{
    factorial *= n;
    n--;
} while (n > 0); резултат в 158-цифри
Console.WriteLine("n! = " + factorial);
```

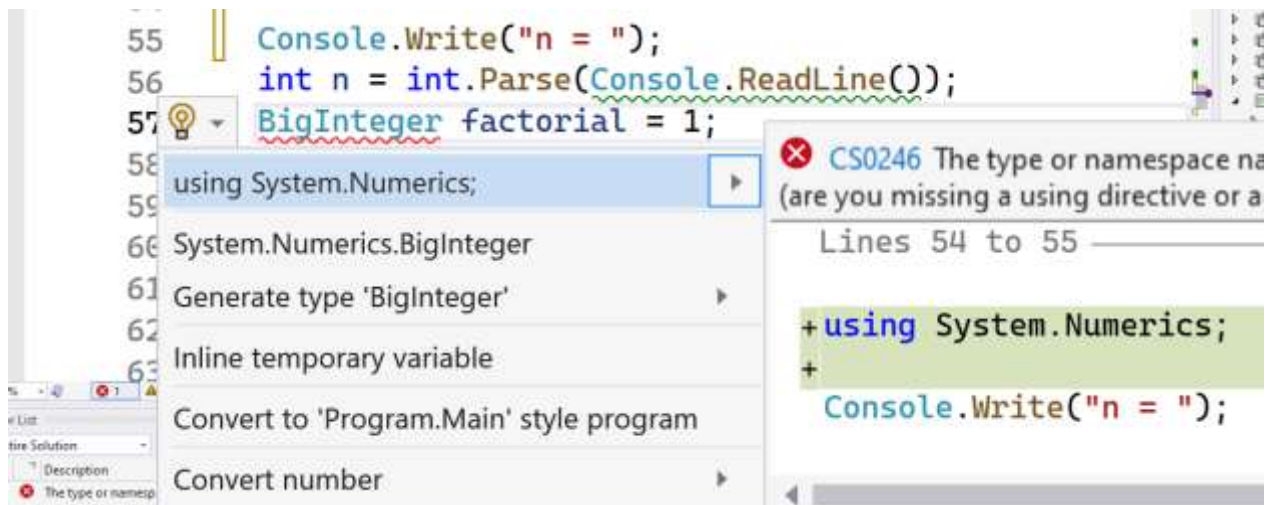
55 `Console.Write("n = ");`
56 `int n = int.Parse(Console.ReadLine());`
57 `BigInteger factorial = 1;`
58 `do`
59 `{`
60 `facto`
61 `n--;`
62 `} while (`
63 `Console.W`

CS0246: The type or namespace name 'BigInteger' could not be found (are you missing a using directive or an assembly reference?)

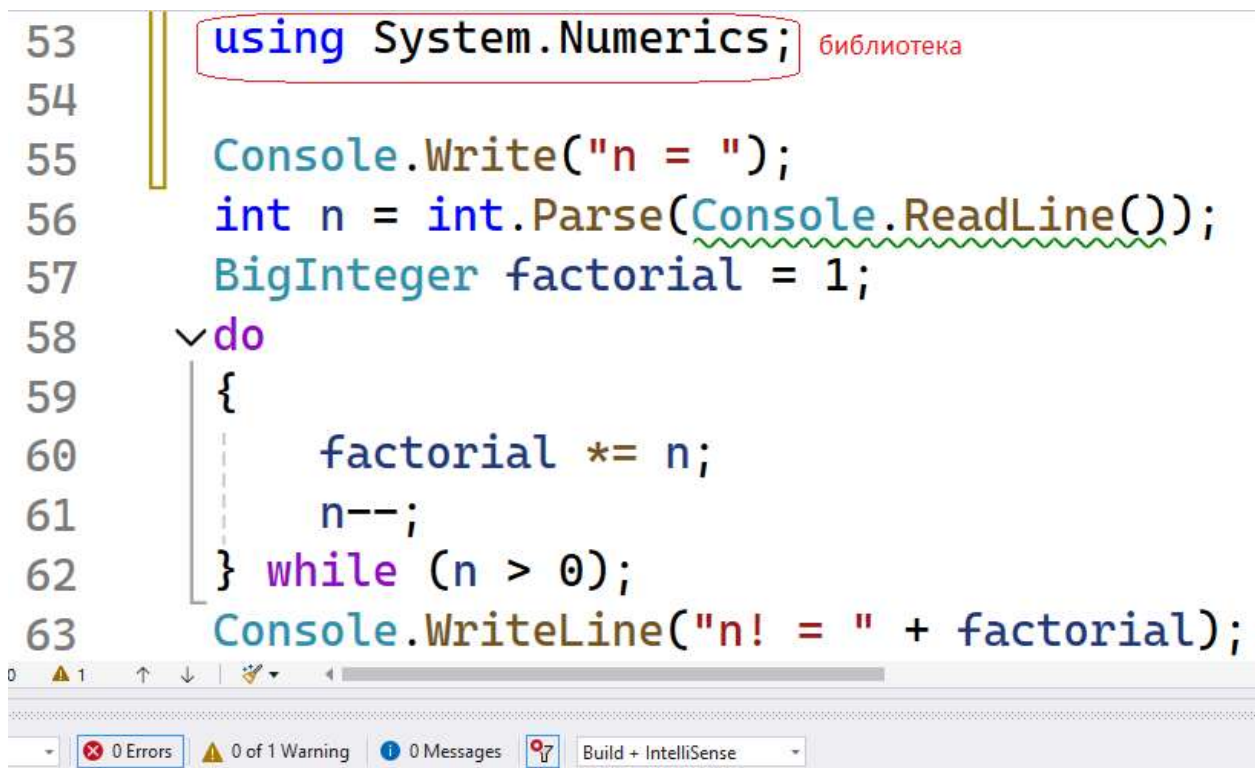
Show potential fixes (Alt

Object	File
while	Program.cs

Ctrl + . -> автокорекция на грешка (предлага решение), в случая, дописва в началото най-отгоре библиотеката, която позволява използване на тип BigInteger:



Всички библиотеки започват с using, последователността им има значение (трябва да са подредени по азбучен ред, ако са повече от 1) и ние извикваме само тези, които ни трябват, защото VS е много тежка програма, с много големи възможности и всяка допълнителна функционалност бави.



Последно: ето още един интересен for() – цикъл, който работи не само с една променлива – i, а и с втора, наречена sum. При този синтаксис имаме разделяне на двете променливи с обикновена запетайка, а условията си остават с ;

```
for (int i = 0, sum = i + 1; i <= 5; i++, sum += i)
{
    Console.WriteLine("i={0}, sum={1}", i, sum);
}
```

И другото интересно нещо е, че може да се използва последователно позициониране на променливите в string със спазване на номерациите 0, 1, 2,... Съответно на тяхно място се вмъкват последователни променливи както е показано. Това е по-старият синтаксис, преди да измислят интерполацията с \$, която е по-удобна и кратка.

```
Microsoft Visual Studio Debug Console
i=0, sum=1
i=1, sum=2
i=2, sum=4
i=3, sum=7
i=4, sum=11
i=5, sum=16
D:\Exercises\11 class\07_while\bin\Debug\net
Press any key to close this window . . .
```

Задача: чрез 1 цикъл for(), да се отпечатаат всички цели числа a, които нарастват от 0 до 5 и всички цели числа b, които намаляват от 10 до 5 и резултатите c, които събират a + b.

Упътване: в условието за край е достатъчна само 1 променлива за 5 цикъла.

```
for(int a = 0, b = 10, c = a + b; a < 5; a++, b--)
{
    Console.WriteLine("c = a + b = ({0} + {1}) = {2}", a, b, c);
}
```

```
Microsoft Visual Studio Debug Console
c = a + b = (0 + 10) = 10
c = a + b = (1 + 9) = 10
c = a + b = (2 + 8) = 10
c = a + b = (3 + 7) = 10
c = a + b = (4 + 6) = 10
D:\Exercises\11 class\07_while\bin\Debug\net
Press any key to close this window . . .
```

По този начин, вместо да използваме 3 вложени цикъла for(), използвахме само 1 с 3 променливи.

Задача за повишаване на оценка: Потребителят въвежда от конзолата информация дотогава, докато не напише „END”. За всяко число, което се дели на 5, се изписва неговото произведение с 3.

```
string input = Console.ReadLine().ToUpper();
```

```
✓ while (input != "END")
```

```
{
```

```
    int n = int.Parse(input);
```

```
✓    if (n % 5 == 0)
```

```
{
```

```
        n *= 3;
```

```
        Console.WriteLine(n);
```

```
    }
```

```
    !!! input = Console.ReadLine().ToUpper();  
}
```

Не забравяйте, че най-важната стъпка след условието за проверка на while, е как да се продължи завъртането, за да не стане цикълът безкраен!!!